

RandomForest

December 30, 2025

1 Random Forest

2 Project: Fraud Detection - Random Forest Model

Mục tiêu: Nâng cao hiệu suất bằng mô hình phi tuyến (Non-linear Model).

2.1 Tại sao chọn Random Forest?

1. **Robustness:** Ít bị ảnh hưởng bởi nhiễu (noise) và outliers hơn Logistic Regression.
2. **Feature Interactions:** Có khả năng tự học các tương tác phức tạp (ví dụ: Giao dịch lớn + Khung giờ lạ).
3. **No Scaling:** Không yêu cầu chuẩn hóa dữ liệu đầu vào.

2.2 Chiến lược Tuning

- **Tập trung vào:** max_depth và min_samples_leaf để kiểm soát Overfitting.
- **Resampling:** Sử dụng SMOTE để cân bằng lại tỷ lệ lớp thiểu số.

```
[1]: ! pip install imbalanced-learn
```

```
Requirement already satisfied: imbalanced-learn in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (0.14.1)
Requirement already satisfied: numpy<3,>=1.25.2 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (2.2.6)
Requirement already satisfied: scipy<2,>=1.11.4 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (1.15.3)
Requirement already satisfied: scikit-learn<2,>=1.4.2 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (1.7.0)
Requirement already satisfied: sklearn-compat<0.2,>=0.1.5 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (0.1.5)
Requirement already satisfied: joblib<2,>=1.2.0 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (1.5.1)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in
```

c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (3.6.0)

[notice] A new release of pip is available: 23.2.1 -> 25.3

[notice] To update, run: python.exe -m pip install --upgrade pip

```
[2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, StratifiedKFold, \
    cross_val_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score,
    average_precision_score, precision_recall_curve,
    precision_score, recall_score, f1_score, roc_curve
)

from imblearn.over_sampling import SMOTE, RandomOverSampler
from imblearn.under_sampling import RandomUnderSampler
from imblearn.combine import SMOTEENN, SMOTETomek
from imblearn.pipeline import Pipeline as ImbPipeline

import optuna
import joblib
import os
```

```
[28]: df = pd.read_csv("C://Users//hi//Downloads//fraud_detection_project//data//
    processed//fraud_feature_engineering.csv")
df
```

```
[28]:
```

	Unnamed: 0	step	type	amount	nameOrig	oldbalanceOrg	\
0	0	401	CASH_IN	290670.88	C1172770329	30613326.59	
1	1	346	CASH_OUT	74969.02	C361253330	0.00	
2	2	34	CASH_OUT	152359.15	C1296741848	0.00	
3	3	302	TRANSFER	553238.82	C765587136	0.00	
4	4	130	CASH_IN	432624.19	C852698921	18910.00	
...	...	...	...	...	...	...	
325929	325929	282	CASH_OUT	34345.29	C943859349	20380.00	

325930	325930	379	PAYMENT	8234.77	C1729134111	272.00
325931	325931	235	CASH_OUT	13499.45	C297276080	0.00
325932	325932	369	CASH_OUT	236678.98	C2094429825	10018.00
325933	325933	395	CASH_OUT	108989.28	C1625687711	337544.00

	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	...	\
0	30903997.48	C1360884032	6252769.10	5962098.22	...	
1	0.00	C1628643300	1510659.40	1585628.42	...	
2	0.00	C1234551068	519498.85	671857.99	...	
3	0.00	C845383981	1278572.18	1831811.01	...	
4	451534.19	C290472837	306529.18	0.00	...	
...	...	...	...	...	...	
325929	0.00	C1322395030	0.00	34345.29	...	
325930	0.00	M1758068394	0.00	0.00	...	
325931	0.00	C1955063491	84106.06	97605.52	...	
325932	0.00	C1407924212	0.00	236678.98	...	
325933	228554.72	C763824199	389833.54	478305.71	...	

	isCustomerDest	hour	day	abs_balance_diff	abs_errorBalanceDest	\
0	1	17	16	581341.77	5.813418e+05	
1	1	10	14	74969.02	1.455192e-11	
2	1	10	1	152359.15	1.000000e-02	
3	1	14	12	553238.82	1.000000e-02	
4	1	10	5	865248.38	7.391534e+05	
...	...	...	...	...	...	
325929	1	18	11	13965.29	0.000000e+00	
325930	0	19	15	7962.77	8.234770e+03	
325931	1	19	9	13499.45	1.000000e-02	
325932	1	9	15	226660.98	0.000000e+00	
325933	1	11	16	0.00	2.051711e+04	

	is_empty_orig	is_empty_dest	amount_ratio_org	log_amount	red_flag
0	0	0	0.009495	12.579950	1
1	1	0	74969.020000	11.224844	1
2	1	0	152359.150000	11.934002	1
3	1	0	553238.820000	13.223547	1
4	0	0	22.876854	12.977627	1
...	...	...	...	...	...
325929	0	1	1.685162	10.444249	1
325930	0	1	30.163993	9.016242	1
325931	1	0	13499.450000	9.510478	1
325932	0	1	23.623014	12.374464	1
325933	0	0	0.322888	11.599014	1

[325934 rows x 25 columns]

```
[29]: df.columns
```

```
[29]: Index(['Unnamed: 0', 'step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg',
          'newbalanceOrig', 'nameDest', 'oldbalanceDest', 'newbalanceDest',
          'isFraud', 'isFlaggedFraud', 'balance_diff', 'errorBalanceDest',
          'isMerchant', 'isCustomerDest', 'hour', 'day', 'abs_balance_diff',
          'abs_errorBalanceDest', 'is_empty_orig', 'is_empty_dest',
          'amount_ratio_org', 'log_amount', 'red_flag'],
          dtype='object')
```

```
[30]: # Drop các cột không cần thiết
drop_cols = [
    "Unnamed: 0",      # index giả
    "nameOrig",        # ID
    "nameDest",        # ID
    "isFlaggedFraud",  # proxy của nhãn
    'oldbalanceOrg',
    'newbalanceOrig',
    'oldbalanceDest',
    'newbalanceDest',
    'step',
    'balance_diff',
    'errorBalanceDest', # NGUỒN LEAKAGE CHÍNH
    'abs_balance_diff',
    'abs_errorBalanceDest',
    'red_flag',         # Derived from leakage
    'amount_ratio_org', # Tín hiệu rút sạch tiền quá lộ liễu
    'is_empty_orig',    # Trạng thái tài khoản ban đầu
    'is_empty_dest'     # Trạng thái tài khoản nhận
]

df = df.drop(columns=[c for c in drop_cols if c in df.columns])

df.columns
```

```
[30]: Index(['type', 'amount', 'isFraud', 'isMerchant', 'isCustomerDest', 'hour',
          'day', 'log_amount'],
          dtype='object')
```

```
[31]: df
```

```
[31]:
```

	type	amount	isFraud	isMerchant	isCustomerDest	hour	day	\
0	CASH_IN	290670.88	0	0	1	17	16	
1	CASH_OUT	74969.02	0	0	1	10	14	
2	CASH_OUT	152359.15	0	0	1	10	1	
3	TRANSFER	553238.82	0	0	1	14	12	
4	CASH_IN	432624.19	0	0	1	10	5	
...	...	...	...	...	...	...	...	
325929	CASH_OUT	34345.29	0	0	1	18	11	

325930	PAYMENT	8234.77	0	1	0	19	15
325931	CASH_OUT	13499.45	0	0	1	19	9
325932	CASH_OUT	236678.98	0	0	1	9	15
325933	CASH_OUT	108989.28	0	0	1	11	16

	log_amount
0	12.579950
1	11.224844
2	11.934002
3	13.223547
4	12.977627
...	...
325929	10.444249
325930	9.016242
325931	9.510478
325932	12.374464
325933	11.599014

[325934 rows x 8 columns]

```
[32]: # Kiểm tra phân phối lớp
print("Phân phối lớp:")
print(df['isFraud'].value_counts())
print(f"Tỷ lệ fraud: {df['isFraud'].mean():.4}")
```

Phân phối lớp:

isFraud

0 317721

1 8213

Name: count, dtype: int64

Tỷ lệ fraud: 0.0252

```
[33]: target = "isFraud"

X = df.drop(columns=target)
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
[34]: print(f"\nKích thước tập train: {X_train.shape}")
print(f"Kích thước tập test: {X_test.shape}")
```

Kích thước tập train: (260747, 7)

Kích thước tập test: (65187, 7)

```
[35]: # Phân loại feature
categorical_features = X.select_dtypes(include=['object', 'category']).columns.
    ↪tolist()
numeric_features = X.select_dtypes(include=['int64', 'float64']).columns.
    ↪tolist()

print(f"\nCategorical features: {categorical_features}")
print(f"Numeric features: {numeric_features}")

# Tạo preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore", drop="first"),
    ↪categorical_features),
        ("num", "passthrough", numeric_features)
    ],
    remainder="drop"
)
```

Categorical features: ['type']

Numeric features: ['amount', 'isMerchant', 'isCustomerDest', 'hour', 'day',
'log_amount']

```
[36]: model = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("clf", RandomForestClassifier(
        n_estimators=300,
        max_depth=None,
        min_samples_leaf=2,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    ))
])
```

```
[37]: model.fit(X_train, y_train)
```

```
[37]: Pipeline(steps=[('preprocess',
    ColumnTransformer(transformers=[('cat',
    OneHotEncoder(drop='first',
    handle_unknown='ignore'),
    ['type']],
    ('num', 'passthrough',
    ['amount', 'isMerchant',
```

```

        'isCustomerDest', 'hour',
        'day', 'log_amount']]])),
    ('clf',
     RandomForestClassifier(class_weight='balanced',
                           min_samples_leaf=2, n_estimators=300,
                           n_jobs=-1, random_state=42)))]

```

```

[38]: y_pred = model.predict(X_test)
      y_prob = model.predict_proba(X_test)[:, 1]

      print("ROC-AUC:", roc_auc_score(y_test, y_prob))
      print("\nConfusion Matrix:")
      print(confusion_matrix(y_test, y_pred))

      print("\nClassification Report:")
      print(classification_report(y_test, y_pred, digits=4))

```

ROC-AUC: 0.9517706480493356

Confusion Matrix:

```

[[63199  345]
 [  579 1064]]

```

Classification Report:

	precision	recall	f1-score	support
0	0.9909	0.9946	0.9927	63544
1	0.7551	0.6476	0.6972	1643
accuracy			0.9858	65187
macro avg	0.8730	0.8211	0.8450	65187
weighted avg	0.9850	0.9858	0.9853	65187

```

[39]: feature_names = model.named_steps["preprocess"].get_feature_names_out()
      importances = model.named_steps["clf"].feature_importances_

      fi = pd.DataFrame({
          "feature": feature_names,
          "importance": importances
      }).sort_values(by="importance", ascending=False)

      fi.head(10)

```

```

[39]:
      feature  importance
4      num__amount  0.194601
9      num__log_amount  0.194250
8      num__day      0.168051

```

7	num__hour	0.149322
3	cat__type_TRANSFER	0.139077
0	cat__type_CASH_OUT	0.060290
6	num__isCustomerDest	0.047177
2	cat__type_PAYMENT	0.026609
5	num__isMerchant	0.019777
1	cat__type_DEBIT	0.000846

2.2.1 Cấu hình Optuna Tuning

Chúng ta sẽ tìm kiếm bộ tham số tối ưu với các ràng buộc sau để tránh Overfitting: * `n_estimators`: 200 - 800 cây. * `max_depth`: Giới hạn từ 5 - 30 (Tránh cây quá sâu sẽ học vẹt dữ liệu train). * `min_samples_leaf`: Tăng lên (2-10) để đảm bảo mỗi lá cây có đủ dữ liệu đại diện.

```
[40]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

def objective(trial):
    rf_params = {
        "n_estimators": trial.suggest_int("n_estimators", 200, 800),
        "max_depth": trial.suggest_int("max_depth", 5, 30),
        "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),
        "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 10),
        "max_features": trial.suggest_float("max_features", 0.3, 1.0),
        "class_weight": "balanced",
        "random_state": 42,
        "n_jobs": -1
    }

    rf = RandomForestClassifier(**rf_params)

    pipe = Pipeline(steps=[
        ("preprocess", preprocessor),
        ("clf", rf)
    ])

    scores = cross_val_score(
        pipe,
        X_train, y_train,
        cv=cv,
        scoring="roc_auc",
        n_jobs=-1
    )

    return scores.mean()
```

```
[41]: study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30)
```

```
print("Best ROC-AUC (CV):", study.best_value)
print("Best params:", study.best_params)
```

```
[I 2025-12-28 22:06:56,092] A new study created in memory with name: no-
name-79f620bd-76d5-4cc9-817b-a828a790840b
[I 2025-12-28 22:08:35,403] Trial 0 finished with value: 0.9645650561106122 and
parameters: {'n_estimators': 319, 'max_depth': 26, 'min_samples_split': 14,
'min_samples_leaf': 2, 'max_features': 0.4611036662064288}. Best is trial 0 with
value: 0.9645650561106122.
[I 2025-12-28 22:11:54,609] Trial 1 finished with value: 0.9657015063168636 and
parameters: {'n_estimators': 617, 'max_depth': 24, 'min_samples_split': 11,
'min_samples_leaf': 8, 'max_features': 0.5716959277059986}. Best is trial 1 with
value: 0.9657015063168636.
[I 2025-12-28 22:15:49,806] Trial 2 finished with value: 0.9680334681824627 and
parameters: {'n_estimators': 559, 'max_depth': 12, 'min_samples_split': 11,
'min_samples_leaf': 9, 'max_features': 0.9224349961245402}. Best is trial 2 with
value: 0.9680334681824627.
[I 2025-12-28 22:23:38,913] Trial 3 finished with value: 0.9676062400980777 and
parameters: {'n_estimators': 731, 'max_depth': 11, 'min_samples_split': 8,
'min_samples_leaf': 2, 'max_features': 0.9182152357754303}. Best is trial 2 with
value: 0.9680334681824627.
[I 2025-12-28 22:28:53,310] Trial 4 finished with value: 0.9673955335317599 and
parameters: {'n_estimators': 529, 'max_depth': 14, 'min_samples_split': 13,
'min_samples_leaf': 2, 'max_features': 0.40298960273217466}. Best is trial 2
with value: 0.9680334681824627.
[I 2025-12-28 22:32:55,248] Trial 5 finished with value: 0.9681445663459829 and
parameters: {'n_estimators': 549, 'max_depth': 8, 'min_samples_split': 4,
'min_samples_leaf': 8, 'max_features': 0.44505015730760433}. Best is trial 5
with value: 0.9681445663459829.
[I 2025-12-28 22:36:48,657] Trial 6 finished with value: 0.9633063384578033 and
parameters: {'n_estimators': 225, 'max_depth': 19, 'min_samples_split': 6,
'min_samples_leaf': 8, 'max_features': 0.8578618580433695}. Best is trial 5 with
value: 0.9681445663459829.
[I 2025-12-28 22:40:45,782] Trial 7 finished with value: 0.9657164309107571 and
parameters: {'n_estimators': 286, 'max_depth': 25, 'min_samples_split': 18,
'min_samples_leaf': 10, 'max_features': 0.6829217706455899}. Best is trial 5
with value: 0.9681445663459829.
[I 2025-12-28 22:46:50,011] Trial 8 finished with value: 0.9701444833857927 and
parameters: {'n_estimators': 565, 'max_depth': 10, 'min_samples_split': 9,
'min_samples_leaf': 5, 'max_features': 0.6698290608736338}. Best is trial 8 with
value: 0.9701444833857927.
[I 2025-12-28 22:49:48,768] Trial 9 finished with value: 0.969590679035109 and
parameters: {'n_estimators': 323, 'max_depth': 9, 'min_samples_split': 4,
'min_samples_leaf': 8, 'max_features': 0.5288539839552101}. Best is trial 8 with
value: 0.9701444833857927.
[I 2025-12-28 22:56:25,497] Trial 10 finished with value: 0.9636184411973086 and
parameters: {'n_estimators': 785, 'max_depth': 5, 'min_samples_split': 20,
```

'min_samples_leaf': 5, 'max_features': 0.7066136650308211}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-28 23:00:43,505] Trial 11 finished with value: 0.963483357701277 and parameters: {'n_estimators': 405, 'max_depth': 17, 'min_samples_split': 2, 'min_samples_leaf': 5, 'max_features': 0.585215602369964}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-28 23:02:55,438] Trial 12 finished with value: 0.9605806637630586 and parameters: {'n_estimators': 415, 'max_depth': 5, 'min_samples_split': 8, 'min_samples_leaf': 6, 'max_features': 0.3079310763614532}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-28 23:08:06,033] Trial 13 finished with value: 0.9701108694944228 and parameters: {'n_estimators': 435, 'max_depth': 9, 'min_samples_split': 7, 'min_samples_leaf': 6, 'max_features': 0.7777616318331048}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-28 23:17:20,384] Trial 14 finished with value: 0.9617829762448682 and parameters: {'n_estimators': 636, 'max_depth': 16, 'min_samples_split': 8, 'min_samples_leaf': 4, 'max_features': 0.7835917961571284}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-28 23:23:20,703] Trial 15 finished with value: 0.9626393304602259 and parameters: {'n_estimators': 446, 'max_depth': 21, 'min_samples_split': 11, 'min_samples_leaf': 6, 'max_features': 0.7863488659265953}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-28 23:39:03,163] Trial 16 finished with value: 0.9613503550153351 and parameters: {'n_estimators': 674, 'max_depth': 30, 'min_samples_split': 6, 'min_samples_leaf': 4, 'max_features': 0.7189523579195755}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-28 23:46:47,060] Trial 17 finished with value: 0.9692384010561174 and parameters: {'n_estimators': 475, 'max_depth': 8, 'min_samples_split': 15, 'min_samples_leaf': 7, 'max_features': 0.7931949896299645}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-28 23:53:37,958] Trial 18 finished with value: 0.9655054501509698 and parameters: {'n_estimators': 378, 'max_depth': 14, 'min_samples_split': 9, 'min_samples_leaf': 4, 'max_features': 0.6179659015605055}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-29 00:03:32,363] Trial 19 finished with value: 0.9679947639827118 and parameters: {'n_estimators': 493, 'max_depth': 11, 'min_samples_split': 6, 'min_samples_leaf': 3, 'max_features': 0.8462054294314814}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-29 00:14:25,305] Trial 20 finished with value: 0.967749149651986 and parameters: {'n_estimators': 609, 'max_depth': 7, 'min_samples_split': 16, 'min_samples_leaf': 1, 'max_features': 0.9822962211353211}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-29 00:17:52,819] Trial 21 finished with value: 0.96957790176289 and parameters: {'n_estimators': 344, 'max_depth': 9, 'min_samples_split': 2, 'min_samples_leaf': 7, 'max_features': 0.5618916286673699}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-29 00:20:03,058] Trial 22 finished with value: 0.9683789490926203 and parameters: {'n_estimators': 201, 'max_depth': 13, 'min_samples_split': 4,

'min_samples_leaf': 7, 'max_features': 0.516239669424782}. Best is trial 8 with value: 0.9701444833857927.

[I 2025-12-29 00:22:56,291] Trial 23 finished with value: 0.9703520210057022 and parameters: {'n_estimators': 252, 'max_depth': 10, 'min_samples_split': 4, 'min_samples_leaf': 10, 'max_features': 0.6487304884741807}. Best is trial 23 with value: 0.9703520210057022.

[I 2025-12-29 00:39:09,910] Trial 24 finished with value: 0.9703666203284396 and parameters: {'n_estimators': 270, 'max_depth': 10, 'min_samples_split': 9, 'min_samples_leaf': 10, 'max_features': 0.6547708176259677}. Best is trial 24 with value: 0.9703666203284396.

[I 2025-12-29 00:41:52,617] Trial 25 finished with value: 0.9665289887965919 and parameters: {'n_estimators': 253, 'max_depth': 15, 'min_samples_split': 13, 'min_samples_leaf': 10, 'max_features': 0.6520755414604201}. Best is trial 24 with value: 0.9703666203284396.

[I 2025-12-29 00:45:13,209] Trial 26 finished with value: 0.9699286690226625 and parameters: {'n_estimators': 267, 'max_depth': 11, 'min_samples_split': 9, 'min_samples_leaf': 9, 'max_features': 0.6669883629262715}. Best is trial 24 with value: 0.9703666203284396.

[I 2025-12-29 00:50:47,160] Trial 27 finished with value: 0.9647002360535064 and parameters: {'n_estimators': 374, 'max_depth': 19, 'min_samples_split': 10, 'min_samples_leaf': 10, 'max_features': 0.7287402742262685}. Best is trial 24 with value: 0.9703666203284396.

[I 2025-12-29 00:53:19,370] Trial 28 finished with value: 0.9673411691950861 and parameters: {'n_estimators': 292, 'max_depth': 7, 'min_samples_split': 5, 'min_samples_leaf': 9, 'max_features': 0.6306811714001748}. Best is trial 24 with value: 0.9703666203284396.

[I 2025-12-29 00:56:12,689] Trial 29 finished with value: 0.9699823395321003 and parameters: {'n_estimators': 333, 'max_depth': 12, 'min_samples_split': 13, 'min_samples_leaf': 9, 'max_features': 0.4934825954727338}. Best is trial 24 with value: 0.9703666203284396.

Best ROC-AUC (CV): 0.9703666203284396

Best params: {'n_estimators': 270, 'max_depth': 10, 'min_samples_split': 9, 'min_samples_leaf': 10, 'max_features': 0.6547708176259677}

```
[42]: best_rf = RandomForestClassifier(
    **study.best_params,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)

best_model = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("clf", best_rf)
])

best_model.fit(X_train, y_train)
```

```
[42]: Pipeline(steps=[('preprocess',
                        ColumnTransformer(transformers=[('cat',
                                                         OneHotEncoder(drop='first',
                                                         handle_unknown='ignore'),
                                                         ['type']),
                                                         ('num', 'passthrough',
                                                         ['amount', 'isMerchant',
                                                         'isCustomerDest', 'hour',
                                                         'day', 'log_amount'])])),
                    ('clf',
                     RandomForestClassifier(class_weight='balanced', max_depth=10,
                                           max_features=0.6547708176259677,
                                           min_samples_leaf=10,
                                           min_samples_split=9, n_estimators=270,
                                           n_jobs=-1, random_state=42)))]
```

```
[43]: y_pred_best = best_model.predict(X_test)
y_prob_best = best_model.predict_proba(X_test)[:, 1]

print("Test ROC-AUC:", roc_auc_score(y_test, y_prob_best))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_best))

print("\nClassification Report:")
print(classification_report(y_test, y_pred_best, digits=4))
```

Test ROC-AUC: 0.9686469831189956

Confusion Matrix:

```
[[58900  4644]
 [  228 1415]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.9961	0.9269	0.9603	63544
1	0.2335	0.8612	0.3674	1643
accuracy			0.9253	65187
macro avg	0.6148	0.8941	0.6639	65187
weighted avg	0.9769	0.9253	0.9453	65187

```
[44]: feature_names = best_model.named_steps["preprocess"].get_feature_names_out()
importances = best_model.named_steps["clf"].feature_importances_

fi = pd.DataFrame({
```

```

    "feature": feature_names,
    "importance": importances
}).sort_values(by="importance", ascending=False)

fi.head(10)

```

```

[44]:
      feature  importance
3  cat__type_TRANSFER    0.220053
4      num__amount    0.169300
9    num__log_amount    0.161338
8      num__day    0.153461
7      num__hour    0.140742
0  cat__type_CASH_OUT    0.115636
6  num__isCustomerDest    0.019758
2  cat__type_PAYMENT    0.011117
5      num__isMerchant    0.008349
1  cat__type_DEBIT    0.000244

```

2.2.2 Phân tích Feature Importance (Random Forest)

Khác với Logistic Regression, Random Forest chỉ ra bức tranh rõ ràng hơn về hành vi:

1. **type_TRANSFER (22%) & amount (17%):** Vẫn là các yếu tố quan trọng nhất.
2. **Sự trôi dạt của biến Thời gian:**
 - day (15.3%) và hour (14.1%) lọt vào Top 5 features quan trọng nhất.
 - **Insight:** Gian lận có tính chu kỳ thời gian (ví dụ: xảy ra vào giờ thấp điểm hoặc các ngày cụ thể) mà mô hình tuyến tính đã bỏ qua.

2.3 THỬ CÁC CHIẾN LƯỢC RESAMPLING

```

[45]: strategies = [
    ('smote', SMOTE(random_state=42)),
    ('smote_enn', SMOTEENN(random_state=42)),
    ('smote_tomek', SMOTETomek(random_state=42)),
    ('oversample', RandomOverSampler(random_state=42)),
    ('undersample', RandomUnderSampler(random_state=42)),
]

```

```

[46]: results = {}

for strategy_name, sampler in strategies:
    print(f"\nĐang thử: {strategy_name.upper()}")

    # Tạo pipeline với resampling
    pipe = ImbPipeline(steps=[
        ("preprocess", preprocessor),
        ("resample", sampler),
        ("clf", RandomForestClassifier(

```

```

        n_estimators=100, # Dùng ít trees để test nhanh
        class_weight=None, # Không dùng vì đã resampling
        random_state=42,
        n_jobs=-1
    ))
])

# Huấn luyện
pipe.fit(X_train, y_train)

# Dự đoán
y_pred = pipe.predict(X_test)
y_prob = pipe.predict_proba(X_test)[:, 1]

# Tính metrics
roc_auc = roc_auc_score(y_test, y_prob)
pr_auc = average_precision_score(y_test, y_prob)
recall = recall_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

# Lưu kết quả
results[strategy_name] = {
    'roc_auc': roc_auc,
    'pr_auc': pr_auc,
    'recall': recall,
    'precision': precision,
    'f1': f1,
    'model': pipe
}

print(f" ROC-AUC: {roc_auc:.4f}")
print(f" PR-AUC: {pr_auc:.4f}")
print(f" Recall: {recall:.4f}")
print(f" Precision: {precision:.4f}")
print(f" F1-Score: {f1:.4f}")

```

Đang thử: SMOTE

ROC-AUC: 0.9352
 PR-AUC: 0.7132
 Recall: 0.6762
 Precision: 0.7049
 F1-Score: 0.6903

Đang thử: SMOTE_ENN

ROC-AUC: 0.9436
 PR-AUC: 0.6580

Recall: 0.7322
Precision: 0.4416
F1-Score: 0.5510

Đang thử: SMOTE_TOMEK

ROC-AUC: 0.9362
PR-AUC: 0.7059
Recall: 0.6792
Precision: 0.6619
F1-Score: 0.6705

Đang thử: OVERSAMPLE

ROC-AUC: 0.9276
PR-AUC: 0.6856
Recall: 0.6360
Precision: 0.7464
F1-Score: 0.6868

Đang thử: UNDERSAMPLE

ROC-AUC: 0.9629
PR-AUC: 0.6067
Recall: 0.8868
Precision: 0.1867
F1-Score: 0.3085

2.4 Chọn chiến lược tốt nhất dựa trên F1-score

```
[47]: best_strategy = max(results.keys(), key=lambda x: results[x]['f1'])  
print(f"\nChiến lược tốt nhất: {best_strategy.upper()} (F1 =   
↪{results[best_strategy]['f1']:.4f})")
```

Chiến lược tốt nhất: SMOTE (F1 = 0.6903)

3 TUNING HYPERPARAMETERS VỚI RESAMPLING TỐT NHẤT

```
[48]: print(f"TUNING RANDOM FOREST VỚI {best_strategy.upper()}")  
  
# Lấy sampler tốt nhất  
best_sampler = strategies[[s[0] for s in strategies].index(best_strategy)][1]  
  
# Hàm objective cho Optuna  
def objective(trial):  
    # Hyperparameters cho Random Forest  
    rf_params = {  
        "n_estimators": trial.suggest_int("n_estimators", 100, 500),
```

```

        "max_depth": trial.suggest_int("max_depth", 10, 50),
        "min_samples_split": trial.suggest_int("min_samples_split", 2, 20),
        "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 10),
        "max_features": trial.suggest_categorical("max_features", ["sqrt",
↪ "log2", None]),
        "bootstrap": trial.suggest_categorical("bootstrap", [True, False]),
        "random_state": 42,
        "n_jobs": -1,
        "class_weight": None # Không dùng vì đã resampling
    }

    # Hyperparameters cho SMOTE (nếu dùng)
    if best_strategy == 'smote':
        k_neighbors = trial.suggest_int("k_neighbors", 3, 10)
        sampling_strategy = trial.suggest_float("sampling_strategy", 0.3, 0.9)
        sampler = SMOTE(
            random_state=42,
            k_neighbors=k_neighbors,
            sampling_strategy=sampling_strategy
        )
    else:
        sampler = best_sampler

    # Tạo pipeline
    pipe = ImbPipeline(steps=[
        ("preprocess", preprocessor),
        ("resample", sampler),
        ("clf", RandomForestClassifier(**rf_params))
    ])

    # Cross-validation
    scores = cross_val_score(
        pipe, X_train, y_train,
        cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=42),
        scoring='roc_auc',
        n_jobs=-1
    )

    return scores.mean()

```

TUNING RANDOM FOREST VỚI SMOTE

3.1 Chạy Optuna

```

[49]: study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=20, show_progress_bar=True)

```

```
print(f"\nBest ROC-AUC (CV): {study.best_value:.6f}")
print(f"Best parameters: {study.best_params}")
```

[I 2025-12-29 01:04:38,529] A new study created in memory with name: no-name-ffed548a-a6a7-42d5-98fa-0c7027dc9a55

0%| | 0/20 [00:00<?, ?it/s]

[I 2025-12-29 01:09:29,328] Trial 0 finished with value: 0.9643575921358255 and parameters: {'n_estimators': 454, 'max_depth': 48, 'min_samples_split': 19, 'min_samples_leaf': 7, 'max_features': 'sqrt', 'bootstrap': False, 'k_neighbors': 6, 'sampling_strategy': 0.42268363417314836}. Best is trial 0 with value: 0.9643575921358255.

[I 2025-12-29 01:15:11,067] Trial 1 finished with value: 0.8878796839082895 and parameters: {'n_estimators': 228, 'max_depth': 35, 'min_samples_split': 10, 'min_samples_leaf': 8, 'max_features': None, 'bootstrap': False, 'k_neighbors': 5, 'sampling_strategy': 0.3288994391892985}. Best is trial 0 with value: 0.9643575921358255.

[I 2025-12-29 01:17:55,277] Trial 2 finished with value: 0.9645614734730592 and parameters: {'n_estimators': 322, 'max_depth': 25, 'min_samples_split': 2, 'min_samples_leaf': 9, 'max_features': 'log2', 'bootstrap': True, 'k_neighbors': 10, 'sampling_strategy': 0.6801982078888964}. Best is trial 2 with value: 0.9645614734730592.

[I 2025-12-29 01:19:34,182] Trial 3 finished with value: 0.9647960191084861 and parameters: {'n_estimators': 237, 'max_depth': 32, 'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt', 'bootstrap': True, 'k_neighbors': 10, 'sampling_strategy': 0.3074522118214229}. Best is trial 3 with value: 0.9647960191084861.

[I 2025-12-29 01:20:56,411] Trial 4 finished with value: 0.9639410370158537 and parameters: {'n_estimators': 218, 'max_depth': 23, 'min_samples_split': 8, 'min_samples_leaf': 9, 'max_features': 'log2', 'bootstrap': False, 'k_neighbors': 10, 'sampling_strategy': 0.5287401687181394}. Best is trial 3 with value: 0.9647960191084861.

[I 2025-12-29 01:22:02,775] Trial 5 finished with value: 0.9628855830800641 and parameters: {'n_estimators': 183, 'max_depth': 23, 'min_samples_split': 14, 'min_samples_leaf': 2, 'max_features': 'log2', 'bootstrap': True, 'k_neighbors': 9, 'sampling_strategy': 0.7926032121432334}. Best is trial 3 with value: 0.9647960191084861.

[I 2025-12-29 01:27:28,153] Trial 6 finished with value: 0.9472208743319562 and parameters: {'n_estimators': 266, 'max_depth': 48, 'min_samples_split': 16, 'min_samples_leaf': 1, 'max_features': None, 'bootstrap': True, 'k_neighbors': 6, 'sampling_strategy': 0.6527966539771011}. Best is trial 3 with value: 0.9647960191084861.

[I 2025-12-29 01:29:45,057] Trial 7 finished with value: 0.9647374593724972 and parameters: {'n_estimators': 161, 'max_depth': 38, 'min_samples_split': 20, 'min_samples_leaf': 9, 'max_features': 'sqrt', 'bootstrap': False, 'k_neighbors': 5, 'sampling_strategy': 0.3212272345265047}. Best is trial 3 with value: 0.9647960191084861.

[I 2025-12-29 01:31:31,265] Trial 8 finished with value: 0.9606874793119263 and

parameters: {'n_estimators': 114, 'max_depth': 47, 'min_samples_split': 18, 'min_samples_leaf': 2, 'max_features': 'sqrt', 'bootstrap': False, 'k_neighbors': 9, 'sampling_strategy': 0.36996633816729474}. Best is trial 3 with value: 0.9647960191084861.

[I 2025-12-29 01:37:33,964] Trial 9 finished with value: 0.9645225215670115 and parameters: {'n_estimators': 458, 'max_depth': 46, 'min_samples_split': 8, 'min_samples_leaf': 5, 'max_features': 'log2', 'bootstrap': True, 'k_neighbors': 6, 'sampling_strategy': 0.8148208260547798}. Best is trial 3 with value: 0.9647960191084861.

[I 2025-12-29 01:40:01,359] Trial 10 finished with value: 0.9676278784744237 and parameters: {'n_estimators': 352, 'max_depth': 10, 'min_samples_split': 3, 'min_samples_leaf': 5, 'max_features': 'sqrt', 'bootstrap': True, 'k_neighbors': 3, 'sampling_strategy': 0.48531304697787647}. Best is trial 10 with value: 0.9676278784744237.

[I 2025-12-29 01:42:43,722] Trial 11 finished with value: 0.9680707021110324 and parameters: {'n_estimators': 349, 'max_depth': 12, 'min_samples_split': 2, 'min_samples_leaf': 5, 'max_features': 'sqrt', 'bootstrap': True, 'k_neighbors': 3, 'sampling_strategy': 0.4850700652246383}. Best is trial 11 with value: 0.9680707021110324.

[I 2025-12-29 01:45:46,174] Trial 12 finished with value: 0.9680662754118797 and parameters: {'n_estimators': 355, 'max_depth': 11, 'min_samples_split': 2, 'min_samples_leaf': 6, 'max_features': 'sqrt', 'bootstrap': True, 'k_neighbors': 3, 'sampling_strategy': 0.5058294460666772}. Best is trial 11 with value: 0.9680707021110324.

[I 2025-12-29 01:48:15,480] Trial 13 finished with value: 0.9674276156547362 and parameters: {'n_estimators': 387, 'max_depth': 10, 'min_samples_split': 5, 'min_samples_leaf': 7, 'max_features': 'sqrt', 'bootstrap': True, 'k_neighbors': 3, 'sampling_strategy': 0.5426349557190282}. Best is trial 11 with value: 0.9680707021110324.

[I 2025-12-29 01:50:37,151] Trial 14 finished with value: 0.9672276311506437 and parameters: {'n_estimators': 398, 'max_depth': 14, 'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt', 'bootstrap': True, 'k_neighbors': 4, 'sampling_strategy': 0.45413398613118644}. Best is trial 11 with value: 0.9680707021110324.

[I 2025-12-29 01:52:49,454] Trial 15 finished with value: 0.9659235484222236 and parameters: {'n_estimators': 300, 'max_depth': 17, 'min_samples_split': 2, 'min_samples_leaf': 6, 'max_features': 'sqrt', 'bootstrap': True, 'k_neighbors': 4, 'sampling_strategy': 0.6029181154142945}. Best is trial 11 with value: 0.9680707021110324.

[I 2025-12-29 02:01:37,727] Trial 16 finished with value: 0.956807810938073 and parameters: {'n_estimators': 400, 'max_depth': 17, 'min_samples_split': 13, 'min_samples_leaf': 4, 'max_features': None, 'bootstrap': True, 'k_neighbors': 3, 'sampling_strategy': 0.560995511394582}. Best is trial 11 with value: 0.9680707021110324.

[I 2025-12-29 02:07:52,548] Trial 17 finished with value: 0.964679966294116 and parameters: {'n_estimators': 484, 'max_depth': 18, 'min_samples_split': 7, 'min_samples_leaf': 6, 'max_features': 'sqrt', 'bootstrap': True, 'k_neighbors': 8, 'sampling_strategy': 0.7399036162838817}. Best is trial 11 with value:

0.9680707021110324.

[I 2025-12-29 02:10:02,976] Trial 18 finished with value: 0.9646466645416364 and parameters: {'n_estimators': 323, 'max_depth': 27, 'min_samples_split': 11, 'min_samples_leaf': 3, 'max_features': 'sqrt', 'bootstrap': True, 'k_neighbors': 4, 'sampling_strategy': 0.4243212900480646}. Best is trial 11 with value: 0.9680707021110324.

[I 2025-12-29 02:19:01,317] Trial 19 finished with value: 0.9592979375285164 and parameters: {'n_estimators': 361, 'max_depth': 13, 'min_samples_split': 4, 'min_samples_leaf': 7, 'max_features': None, 'bootstrap': True, 'k_neighbors': 7, 'sampling_strategy': 0.6271108885404649}. Best is trial 11 with value: 0.9680707021110324.

Best ROC-AUC (CV): 0.968071

Best parameters: {'n_estimators': 349, 'max_depth': 12, 'min_samples_split': 2, 'min_samples_leaf': 5, 'max_features': 'sqrt', 'bootstrap': True, 'k_neighbors': 3, 'sampling_strategy': 0.4850700652246383}

4 HUẤN LUYỆN MÔ HÌNH CUỐI CÙNG

```
[50]: # Lấy best parameters
best_params = study.best_params

# Tạo sampler với best parameters (nếu có)
if best_strategy == 'smote':
    sampler = SMOTE(
        random_state=42,
        k_neighbors=best_params.get('k_neighbors', 5),
        sampling_strategy=best_params.get('sampling_strategy', 0.5)
    )
else:
    sampler = best_sampler

# Tạo Random Forest với best parameters
rf_params = {
    'n_estimators': best_params.get('n_estimators', 300),
    'max_depth': best_params.get('max_depth', None),
    'min_samples_split': best_params.get('min_samples_split', 2),
    'min_samples_leaf': best_params.get('min_samples_leaf', 1),
    'max_features': best_params.get('max_features', 'sqrt'),
    'bootstrap': best_params.get('bootstrap', True),
    'random_state': 42,
    'n_jobs': -1,
    'class_weight': None
}
```

```
[51]: # Tạo final pipeline
final_pipe = ImbPipeline(steps=[
    ("preprocess", preprocessor),
    ("resample", sampler),
    ("clf", RandomForestClassifier(**rf_params))
])
```

```
[52]: # Huấn luyện
final_pipe.fit(X_train, y_train)
```

```
[52]: Pipeline(steps=[('preprocess',
                        ColumnTransformer(transformers=[('cat',
                                                         OneHotEncoder(drop='first',
                                                         handle_unknown='ignore'),
                                                         ['type']),
                                                         ('num', 'passthrough',
                                                         ['amount', 'isMerchant',
                                                         'isCustomerDest', 'hour',
                                                         'day', 'log_amount'])])),
                        ('resample',
                         SMOTE(k_neighbors=3, random_state=42,
                               sampling_strategy=0.4850700652246383)),
                        ('clf',
                         RandomForestClassifier(max_depth=12, min_samples_leaf=5,
                                                 n_estimators=349, n_jobs=-1,
                                                 random_state=42))])
```

5 TUNING THRESHOLD

```
[53]: # Lấy probabilities
y_prob_final = final_pipe.predict_proba(X_test)[: , 1]

# Tìm threshold tối ưu
precisions, recalls, thresholds = precision_recall_curve(y_test, y_prob_final)
f1_scores = 2 * (precisions[:-1] * recalls[:-1]) / (precisions[:-1] + recalls[
    ↪-1] + 1e-10)

# Threshold với F1 cao nhất
best_idx = np.argmax(f1_scores)
best_threshold = thresholds[best_idx]
best_f1 = f1_scores[best_idx]

# Threshold với recall cao (>= 0.9) và precision tốt nhất
high_recall_indices = np.where(recalls[:-1] >= 0.9)[0]
if len(high_recall_indices) > 0:
    best_high_recall_idx = high_recall_indices[np.
    ↪argmax(precisions[high_recall_indices])]
```

```

        high_recall_threshold = thresholds[best_high_recall_idx]
    else:
        high_recall_threshold = best_threshold

    print(f"Best threshold (max F1): {best_threshold:.4f} (F1 = {best_f1:.4f})")
    print(f"High-recall threshold (recall >= 0.9): {high_recall_threshold:.4f}")

    # Dùng threshold tối ưu
    y_pred_final = (y_prob_final >= best_threshold).astype(int)

```

Best threshold (max F1): 0.6348 (F1 = 0.6966)

High-recall threshold (recall >= 0.9): 0.1049

6 ĐÁNH GIÁ MÔ HÌNH CUỐI

```

[54]: # Tính metrics
roc_auc_final = roc_auc_score(y_test, y_prob_final)
pr_auc_final = average_precision_score(y_test, y_prob_final)
recall_final = recall_score(y_test, y_pred_final)
precision_final = precision_score(y_test, y_pred_final)
f1_final = f1_score(y_test, y_pred_final)

print("Classification Report:")
print(classification_report(y_test, y_pred_final, digits=4))

print(f"\nROC-AUC: {roc_auc_final:.6f}")
print(f"PR-AUC: {pr_auc_final:.6f}")
print(f"Recall: {recall_final:.4f}")
print(f"Precision: {precision_final:.4f}")
print(f"F1-Score: {f1_final:.4f}")

print("\nConfusion Matrix:")
cm_final = confusion_matrix(y_test, y_pred_final)
print(cm_final)

```

Classification Report:

	precision	recall	f1-score	support
0	0.9907	0.9950	0.9928	63544
1	0.7682	0.6372	0.6966	1643
accuracy			0.9860	65187
macro avg	0.8794	0.8161	0.8447	65187
weighted avg	0.9851	0.9860	0.9854	65187

ROC-AUC: 0.966421

PR-AUC: 0.732095
Recall: 0.6372
Precision: 0.7682
F1-Score: 0.6966

Confusion Matrix:
[[63228 316]
[596 1047]]

7 SO SÁNH KẾT QUẢ

```
[55]: # Tính metrics
roc_auc_baseline = roc_auc_score(y_test, y_prob)
pr_auc_baseline = average_precision_score(y_test, y_prob)
recall_baseline = recall_score(y_test, y_pred)
precision_baseline = precision_score(y_test, y_pred)
f1_baseline = f1_score(y_test, y_pred)

print(f"ROC-AUC: {roc_auc_baseline:.6f}")
print(f"PR-AUC: {pr_auc_baseline:.6f}")
print(f"Recall: {recall_baseline:.4f}")
print(f"Precision: {precision_baseline:.4f}")
print(f"F1-Score: {f1_baseline:.4f}")
```

ROC-AUC: 0.962908
PR-AUC: 0.606704
Recall: 0.8868
Precision: 0.1867
F1-Score: 0.3085

```
[56]: comparison = pd.DataFrame({
    'Metric': ['ROC-AUC', 'PR-AUC', 'Recall', 'Precision', 'F1-Score'],
    'Baseline (no resampling)': [
        f'{roc_auc_baseline:.6f}',
        f'{pr_auc_baseline:.6f}',
        f'{recall_baseline:.4f}',
        f'{precision_baseline:.4f}',
        f'{f1_baseline:.4f}'
    ],
    f'With {best_strategy.upper()} + Tuning': [
        f'{roc_auc_final:.6f}',
        f'{pr_auc_final:.6f}',
        f'{recall_final:.4f}',
        f'{precision_final:.4f}',
        f'{f1_final:.4f}'
    ],
    'Improvement': [
```

```

        f'{{{roc_auc_final - roc_auc_baseline} / roc_auc_baseline * 100}:.2f}% ',
        f'{{{pr_auc_final - pr_auc_baseline} / pr_auc_baseline * 100}:.2f}% ',
        f'{{{recall_final - recall_baseline} / recall_baseline * 100}:.2f}% ',
        f'{{{precision_final - precision_baseline} / precision_baseline * 100}:.
↪2f}% ',
        f'{{{f1_final - f1_baseline} / f1_baseline * 100}:.2f}% '
    ]
})

print(comparison.to_string(index=False))

```

	Metric	Baseline (no resampling)	With SMOTE + Tuning	Improvement
R	ROC-AUC	0.962908	0.966421	0.36%
P	PR-AUC	0.606704	0.732095	20.67%
R	Recall	0.8868	0.6372	-28.14%
P	Precision	0.1867	0.7682	311.39%
F	F1-Score	0.3085	0.6966	125.81%

8 VỀ BIỂU ĐỒ

```

[57]: fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# ROC Curve
fpr_baseline, tpr_baseline, _ = roc_curve(y_test, y_prob)
fpr_final, tpr_final, _ = roc_curve(y_test, y_prob_final)

axes[0, 0].plot(fpr_baseline, tpr_baseline, label=f'Baseline (AUC =
↪{{{roc_auc_baseline:.3f}}}', linewidth=2)
axes[0, 0].plot(fpr_final, tpr_final, label=f'Final (AUC = {{{roc_auc_final:.
↪3f}}}', linewidth=2)
axes[0, 0].plot([0, 1], [0, 1], 'k--', alpha=0.5)
axes[0, 0].set_xlabel('False Positive Rate', fontsize=12)
axes[0, 0].set_ylabel('True Positive Rate', fontsize=12)
axes[0, 0].set_title('ROC Curves Comparison', fontsize=14, fontweight='bold')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Precision-Recall Curve
precision_baseline_curve, recall_baseline_curve, _ =
↪precision_recall_curve(y_test, y_prob)
precision_final_curve, recall_final_curve, _ = precision_recall_curve(y_test,
↪y_prob_final)

axes[0, 1].plot(recall_baseline_curve, precision_baseline_curve,
                label=f'Baseline (AP = {{{pr_auc_baseline:.3f}}}', linewidth=2)
axes[0, 1].plot(recall_final_curve, precision_final_curve,

```

```

        label=f'Final (AP = {pr_auc_final:.3f})', linewidth=2)
axes[0, 1].set_xlabel('Recall', fontsize=12)
axes[0, 1].set_ylabel('Precision', fontsize=12)
axes[0, 1].set_title('Precision-Recall Curves', fontsize=14, fontweight='bold')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Confusion Matrix (Final)
sns.heatmap(cm_final, annot=True, fmt='d', cmap='Blues', ax=axes[0, 2])
axes[0, 2].set_xlabel('Predicted', fontsize=12)
axes[0, 2].set_ylabel('Actual', fontsize=12)
axes[0, 2].set_title('Confusion Matrix (Final Model)', fontsize=14,
    ↪fontweight='bold')

# Threshold Analysis
axes[1, 0].plot(thresholds, f1_scores, 'b-', linewidth=2)
axes[1, 0].axvline(x=best_threshold, color='r', linestyle='--', label=f'Best_
    ↪threshold = {best_threshold:.3f}')
axes[1, 0].set_xlabel('Threshold', fontsize=12)
axes[1, 0].set_ylabel('F1-Score', fontsize=12)
axes[1, 0].set_title('Threshold vs F1-Score', fontsize=14, fontweight='bold')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Feature Importance
feature_names = final_pipe.named_steps['preprocess'].get_feature_names_out()
importances = final_pipe.named_steps['clf'].feature_importances_

feature_importance_df = pd.DataFrame({
    'feature': feature_names,
    'importance': importances
}).sort_values('importance', ascending=False).head(15)

axes[1, 1].barh(range(len(feature_importance_df)),
    ↪feature_importance_df['importance'])
axes[1, 1].set_yticks(range(len(feature_importance_df)))
axes[1, 1].set_yticklabels(feature_importance_df['feature'])
axes[1, 1].set_xlabel('Importance', fontsize=12)
axes[1, 1].set_title('Top 15 Feature Importances', fontsize=14,
    ↪fontweight='bold')
axes[1, 1].grid(True, alpha=0.3, axis='x')

# Comparison Bar Chart
metrics = ['ROC-AUC', 'PR-AUC', 'Recall', 'Precision', 'F1-Score']
baseline_values = [roc_auc_baseline, pr_auc_baseline, recall_baseline,
    ↪precision_baseline, f1_baseline]

```

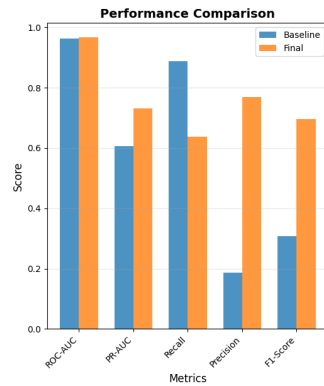
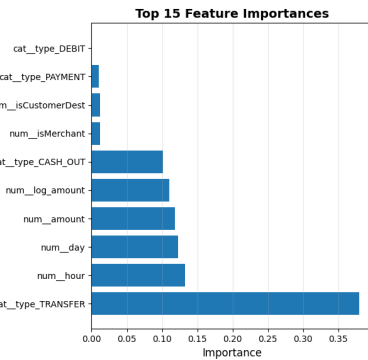
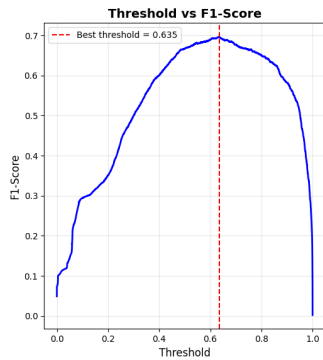
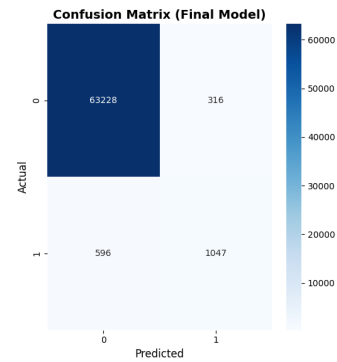
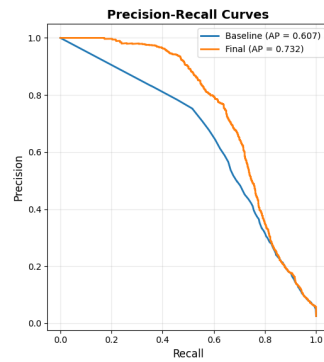
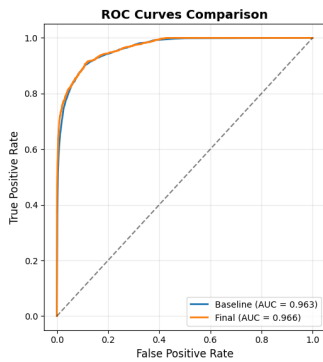
```

final_values = [roc_auc_final, pr_auc_final, recall_final, precision_final,
                f1_final]

x = np.arange(len(metrics))
width = 0.35
axes[1, 2].bar(x - width/2, baseline_values, width, label='Baseline', alpha=0.8)
axes[1, 2].bar(x + width/2, final_values, width, label='Final', alpha=0.8)
axes[1, 2].set_xlabel('Metrics', fontsize=12)
axes[1, 2].set_ylabel('Score', fontsize=12)
axes[1, 2].set_title('Performance Comparison', fontsize=14, fontweight='bold')
axes[1, 2].set_xticks(x)
axes[1, 2].set_xticklabels(metrics, rotation=45, ha='right')
axes[1, 2].legend()
axes[1, 2].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

```



9 THỬ NGHIỆM VỚI THRESHOLD KHÁC NHAU

```
[58]: thresholds_to_try = [0.1, 0.2, 0.3, 0.4, 0.5, best_threshold,
    ↪ high_recall_threshold]
results_thresholds = []

for thr in thresholds_to_try:
    y_pred_thr = (y_prob_final >= thr).astype(int)
    recall_thr = recall_score(y_test, y_pred_thr)
    precision_thr = precision_score(y_test, y_pred_thr)
    f1_thr = f1_score(y_test, y_pred_thr)

    results_thresholds.append({
        'threshold': thr,
        'recall': recall_thr,
        'precision': precision_thr,
        'f1': f1_thr
    })

    print(f"Threshold {thr:.3f}: Recall = {recall_thr:.4f}, Precision =
    ↪ {precision_thr:.4f}, F1 = {f1_thr:.4f}")

# Chuyển sang DataFrame
threshold_results_df = pd.DataFrame(results_thresholds)
print("\nBảng so sánh thresholds:")
print(threshold_results_df.to_string(index=False))
```

```
Threshold 0.100: Recall = 0.9057, Precision = 0.1759, F1 = 0.2947
Threshold 0.200: Recall = 0.8594, Precision = 0.2203, F1 = 0.3507
Threshold 0.300: Recall = 0.7949, Precision = 0.3642, F1 = 0.4995
Threshold 0.400: Recall = 0.7468, Precision = 0.5020, F1 = 0.6004
Threshold 0.500: Recall = 0.7079, Precision = 0.6314, F1 = 0.6674
Threshold 0.635: Recall = 0.6372, Precision = 0.7682, F1 = 0.6966
Threshold 0.105: Recall = 0.9002, Precision = 0.1770, F1 = 0.2959
```

Bảng so sánh thresholds:

threshold	recall	precision	f1
0.100000	0.905660	0.175949	0.294653
0.200000	0.859404	0.220315	0.350720
0.300000	0.794887	0.364194	0.499522
0.400000	0.746805	0.502046	0.600440
0.500000	0.707851	0.631379	0.667432
0.634812	0.637249	0.768158	0.696607
0.104924	0.900183	0.177020	0.295859

10 LƯU MÔ HÌNH VÀ KẾT QUẢ

```
[60]: # Tạo thư mục
os.makedirs('models', exist_ok=True)

# Lưu model
model_path = 'C:/Users/hi/Downloads/fraud_detection_project/models/
↳random_forest_final.pkl'
joblib.dump(final_pipe, model_path)
print(f" Model đã được lưu tại: {model_path}")

# Lưu threshold
threshold_data = {
    'best_threshold': best_threshold,
    'high_recall_threshold': high_recall_threshold,
    'best_f1': best_f1
}
threshold_path = 'C:/Users/hi/Downloads/fraud_detection_project/models/
↳rf_best_threshold.pkl'
joblib.dump(threshold_data, threshold_path)
print(f" Threshold data đã được lưu tại: {threshold_path}")

# Lưu feature importance
feature_importance_path = 'C:/Users/hi/Downloads/fraud_detection_project/models/
↳rf_feature_importance.csv'
feature_importance_df.to_csv(feature_importance_path, index=False)
print(f" Feature importance đã được lưu tại: {feature_importance_path}")

# Lưu kết quả đánh giá
eval_results = {
    'roc_auc': roc_auc_final,
    'pr_auc': pr_auc_final,
    'recall': recall_final,
    'precision': precision_final,
    'f1': f1_final,
    'confusion_matrix': cm_final.tolist()
}
eval_path = 'C:/Users/hi/Downloads/fraud_detection_project/models/
↳rf_evaluation_results.pkl'
joblib.dump(eval_results, eval_path)
print(f" Evaluation results đã được lưu tại: {eval_path}")

# Lưu best parameters
params_path = 'C:/Users/hi/Downloads/fraud_detection_project/models/
↳rf_best_params.pkl'
joblib.dump(best_params, params_path)
```

```

print(f" Best parameters đã được lưu tại: {params_path}")

print("XUẤT BÁO CÁO TÓM TẮT")

print(f"""
TÓM TẮT KẾT QUẢ RANDOM FOREST:
-----
1. Chiến lược resampling tốt nhất: {best_strategy.upper()}
2. Số lượng cây (n_estimators): {best_params.get('n_estimators', 300)}
3. Độ sâu tối đa (max_depth): {best_params.get('max_depth', 'Không giới hạn')}
4. Threshold tối ưu: {best_threshold:.4f}
5. Threshold cho recall cao: {high_recall_threshold:.4f}

KẾT QUẢ ĐÁNH GIÁ:
-----
- ROC-AUC: {roc_auc_final:.6f} ({((roc_auc_final - roc_auc_baseline)/
↪roc_auc_baseline*100):+.2f}% so với baseline)
- PR-AUC: {pr_auc_final:.6f} ({((pr_auc_final - pr_auc_baseline)/
↪pr_auc_baseline*100):+.2f}% so với baseline)
- Recall: {recall_final:.4f} ({((recall_final - recall_baseline)/
↪recall_baseline*100):+.2f}% so với baseline)
- Precision: {precision_final:.4f} ({((precision_final - precision_baseline)/
↪precision_baseline*100):+.2f}% so với baseline)
- F1-Score: {f1_final:.4f} ({((f1_final - f1_baseline)/f1_baseline*100):+.2f}%
↪so với baseline)

FEATURES QUAN TRỌNG NHẤT:
-----
{feature_importance_df.head(5).to_string(index=False)}
""")

```

Model đã được lưu tại:
C:/Users/hi/Downloads/fraud_detection_project/models/random_forest_final.pkl
Threshold data đã được lưu tại:
C:/Users/hi/Downloads/fraud_detection_project/models/rf_best_threshold.pkl
Feature importance đã được lưu tại:
C:/Users/hi/Downloads/fraud_detection_project/models/rf_feature_importance.csv
Evaluation results đã được lưu tại:
C:/Users/hi/Downloads/fraud_detection_project/models/rf_evaluation_results.pkl
Best parameters đã được lưu tại:
C:/Users/hi/Downloads/fraud_detection_project/models/rf_best_params.pkl
XUẤT BÁO CÁO TÓM TẮT

TÓM TẮT KẾT QUẢ RANDOM FOREST:

1. Chiến lược resampling tốt nhất: SMOTE
2. Số lượng cây (n_estimators): 349
3. Độ sâu tối đa (max_depth): 12
4. Threshold tối ưu: 0.6348
5. Threshold cho recall cao: 0.1049

KẾT QUẢ ĐÁNH GIÁ:

-
- ROC-AUC: 0.966421 (+0.36% so với baseline)
 - PR-AUC: 0.732095 (+20.67% so với baseline)
 - Recall: 0.6372 (-28.14% so với baseline)
 - Precision: 0.7682 (+311.39% so với baseline)
 - F1-Score: 0.6966 (+125.81% so với baseline)

FEATURES QUAN TRỌNG NHẤT:

feature	importance
cat__type_TRANSFER	0.380119
num__hour	0.132722
num__day	0.122304
num__amount	0.118463
num__log_amount	0.110424

10.0.1 Kết luận Random Forest

- **Best Strategy:** SMOTE (Regular).
- **Kết quả:** F1-Score đạt **0.6966**.
- **So sánh:**
 - Tốt hơn Logistic Regression (0.56) rất nhiều.
 - Precision đạt **0.7682**, giảm đáng kể số lượng báo động giả (False Positives) so với Logistic Regression (chỉ 0.57).